

3. Huffman Coding

Storage space for files can be saved by compressing them, i.e. by replacing each symbol by a unique binary string.

Here the codewords can differ in length. Then they need to be *prefix-free* in the sense that no codeword is a prefix of another code. Otherwise, decoding is impossible.

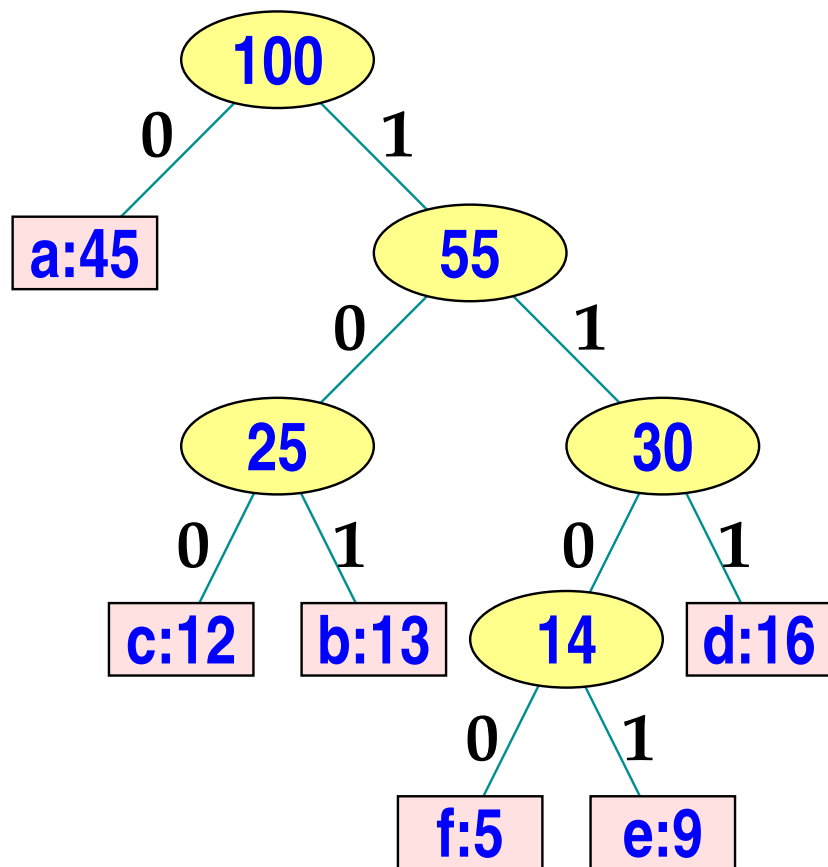
The **character coding problem** is the problem of finding, given an alphabet $C = \{a_1, \dots, a_n\}$ and its frequencies $f_1, \dots, f_n$, a set of prefix-free binary code $W = [w_1, \dots, w_n]$ that minimizes the average code length

$$\sum_{i=1}^n f_i \cdot |w_i|.$$

Depict a prefix-free binary code using a binary tree, where each left branch corresponds to the bit 0, each right branch corresponds to the bit 1, and the leaves are uniquely labeled by the symbols in C .

The codeword of a symbol a in C is the concatenation of the edge labels that are encountered on the path from the root to a .

Each node v is labeled by the frequency sum of the symbols in $subtree(v)$.

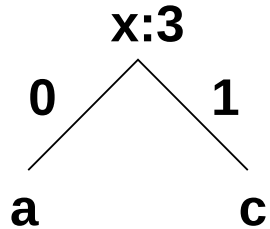


The Huffman coding is a greedy method for obtaining an optimal prefix-free binary code, which uses the following idea: For each i , $1 \leq i \leq n$, create a leaf node v_i corresponding to a_i having frequency f_i . Let $D = \{v_1, \dots, v_n\}$. Repeat the following until $\|D\| = 1$.

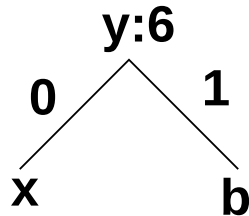
- Select from D the **two nodes with the lowest frequencies**. Call them x and y .
 - Create a node z having x as the left child and y as the right child.
 - Set $f[z]$ to $f[x] + f[y]$.
 - Remove x and y from D and add z to D .
- ★ The replacement will force the codeword of x (y) to be that of z followed by a 0 (a 1).

An example: a:1, b:3, c:2, d:4, e:5

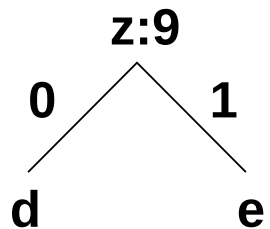
1. $a \& c \rightarrow x$:



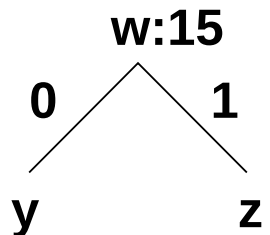
2. $x \& b \rightarrow y$:



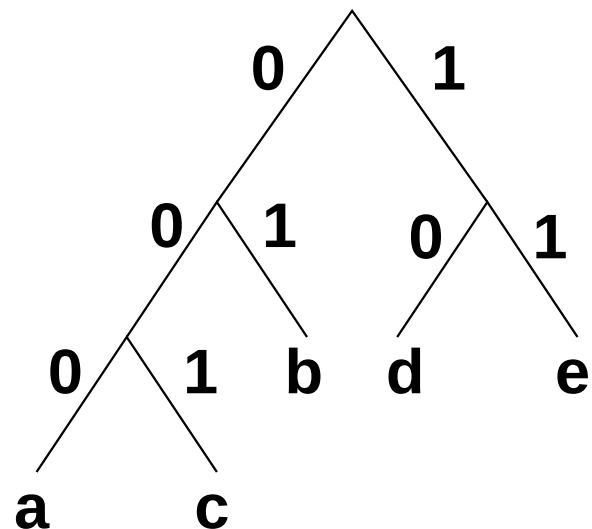
3. $d \& e \rightarrow z$:



4. $y \& z \rightarrow w$:



The resulting tree



The idea can be implemented using a priority-queue that is keyed on f .

The Correctness of The Greedy Method

Lemma B If x and y have the lowest frequencies in an alphabet C , then C has an optimal prefix code in which x and y are siblings.

Proof Let T be an optimal code and let h be the height of T . There are two leaves at depth h that are siblings. If they are not x and y , exchange the nodes with x and y . This will not increase the average code length. ■

Lemma C Create an alphabet D from C by replacing x and y by a single letter z such that $f[z] = f[x] + f[y]$. Then there exists a one-to-one correspondence between

- the set of code trees for D in which z is a leaf and
- the set of code trees for C in which x and y are siblings.

Proof By contradiction:

$B(T)$ is cost of tree for C , $B(T')$ is cost of tree for D .

$$B(T) = B(T') + f[x] + f[y]$$

$$B(T') = B(T) - f[x] - f[y]$$

Suppose $B(T'') < B(T)$.

Create T''' by merging x and y in T'' .

$$\begin{aligned} B(T''') &= B(T'') - f[x] - f[y] \\ &< B(T) - f[x] - f[y] \\ &= B(T') \end{aligned}$$

Suppose that x and y are letters with the lowest frequencies in C . Obtain an optimal code T for D and replace z by a depth-one binary tree with x and y as the leaves. Then we obtain an optimal code for C .